第十届

全国大学生集成电路创新创业大赛

报告类型：作品报告

参赛杯赛：“算能杯”企业命题

作品名称：PicoClaw 视障 Agent：基于 MaixCam 的实时盲道检测

队伍编号：CICC1000580

团队名称：汉 buger 队

作品核心亮点

1 自主解析模型输出节点架构

在无官方适配的情况下，独立完成 YOLO26n-seg 检测头输出节点的识别与分类

背景

YOLO26n-seg 为较新模型，Sipeed 方及 TPU-MLIR 工具链均未提供该模型的后处理示例与 API 支持。本项目通过自主分析模型结构，完成了输出节点的完整解析。

节点识别

通过 Netron 可视化分析 ONNX 模型结构，逐层追踪 model.23 检测头的输出分支，识别出 3 个尺度（28×40、14×20、7×10）下的三类输出，共计 10 个输出节点。

通道数分类路由

编写自动分类逻辑，按输出张量的通道维度（4/32/1）自动将其路由到对应的解码分支，实现对不同输出节点的精准处理。

输出类型	通道数	3 个尺度的形状	节点数
检测框回归	4	(1,4,28,40) · (1,4,14,20) · (1,4,7,10)	3
原型特征	32	(1,32,28,40) · (1,32,14,20) · (1,32,7,10)	3
掩码系数	1	(1,1,28,40) · (1,1,14,20) · (1,1,7,10)	3
全局分割输出	32	(1,32,56,80)	1

> ◆ **意义：**该工作使得 YOLO26n-seg 模型可以在 TPU-MLIR 工具链尚未官方适配的情况下完成端到端部署，具有通用性和可移植性，可为同类新模型的端侧部署提供参考。

2 自研究完整后处理链路

在 Sipeed 官方未支持的前提下，自行实现从模型原始输出到盲道方向判定的全链路后处理

背景

Sipeed 官方未提供 YOLO26-seg 实例分割模型的后处理教程与 API，本项目基于 MaixPy（Python）和 MaixCDK（C）两种框架，自行编写了完整的后处理代码。

方向判定算法流程

01 裁剪掩码**下半区域**（聚焦近处盲道）

02 将掩码水平三等分为 **左/中/右** 区域，统计各区域掩码像素占比

03 三等分投票初步判定方向倾向

04 计算掩码**全局质心水平偏移比**，精细修正方向

05 输出 `LEFT` `RIGHT` `CENTER` `CENTER-LEFT` `CENTER-RIGHT` `WEAK` `NOT FOUND`

> ◆ **意义：**实现了掩码生成、多级降级阈值策略等完整后处理链路，并提供 MaixPy 与 MaixCDK 两种版本，填补了 Sipeed 生态中 YOLO26-seg 后处理的空白。

3 PicoClaw 端侧 Agent 框架集成

首次在 MaixCam RISC-V 平台上部署 PicoClaw 超轻量 AI Agent 框架，实现端侧智能体能力

背景

传统端侧视觉方案仅能完成本地推理，缺乏与用户/监护人的**实时远程交互**能力。PicoClaw 是一个内存占用 `<10MB` 的超轻量 AI Agent 框架，原生支持 RISC-V 架构，具备 Skill 扩展机制和多通道消息能力（微信、飞书、钉钉等），为端侧 Agent 化提供了理想方案。

集成架构

01 在 MaixCam 上部署 PicoClaw **riscv64** 版本，配置微信通道

02 开发两个自定义 Skill：`盲道检测 Skill` 和 `拍照识别 Skill`

03 PicoClaw Agent 统一调度 Skill，通过微信通道实现**双向交互**

组件	功能	实现方式
PicoClaw Agent	Skill 调度、消息路由、通道管理	PicoClaw riscv64 版本
盲道检测 Skill	封装推理管线，持续检测并推送方向	自定义 Skill（Python）
拍照识别 Skill	暂停检测、拍照、障碍物识别、结果推送	自定义 Skill（Python）
微信通道	用户指令接收与结果消息推送	PicoClaw 通道适配

> ◆ **意义：**将端侧推理能力与 Agent 交互能力相结合，使 MaixCam 从单纯的"推理终端"升级为具备远程交互能力的"智能体"，为视障辅助场景提供了"感知-决策-交互"的完整闭环。

4 双 Skill 协同设计与微信远程控制

自定义实现两个 Skill，覆盖"持续导航"和"障碍物识别"两大视障出行核心场景，通过微信实现远程控制

盲道检测 Skill（持续运行）

01 摄像头实时采集画面

02 TPU INT8 推理 → 后处理 → 方向判定

03 方向结果通过 PicoClaw Agent 推送至**微信**

拍照识别 Skill（按需触发）

01 用户通过微信发送 `拍照` 指令

02 暂停盲道检测 → 拍摄当前画面

03 障碍物识别分析 → 结果推送至微信 → 恢复检测

微信控制指令

用户指令	响应动作
开始检测	启动盲道检测 Skill，持续推送方向信息
停止检测	停止盲道检测 Skill
拍照 / 看看前面	暂停检测 → 拍照 → 识别 → 推送结果 → 恢复检测

> ◆ **意义：**用户或监护人无需专业知识，仅通过微信即可远程控制系统的运行状态和功能切换，极大降低了视障辅助设备的使用门槛，同时为远程监护提供了实时信息通道。

一、系统工作原理与关键技术原理分析

1.1 项目背景

盲道是视障人士出行的重要基础设施，但实际环境中盲道常被占用、损坏或难以辨识。基于 AI 视觉的盲道检测系统可以帮助视障人士实时感知盲道方向，但此类系统需要在低功耗的端侧设备上运行，对模型的推理速度和资源占用提出了严格要求。

本项目在算能 MaixCam（CV181X）端侧平台上，实现一个实时盲道实例分割系统，要求：

- 推理帧率 ≥ 15 FPS（满足实时交互需求）
- 分割精度满足实际导航要求
- 模型体积适合端侧存储

然而，单纯的盲道检测方向判定仍不足以满足视障人士的实际出行需求。在真实场景中，视障人士还需要：（1）与外界进行实时交互，将检测结果及时反馈给用户或远程监护人；（2）在遇到障碍物时能够快速识别障碍物类型，辅助决策是否绕行；（3）通过便捷的通信渠道实现远程控制与信息获取。

为此，本项目在端侧推理的基础上，进一步集成了 PicoClaw——一个超轻量级 AI Agent 框架，通过自定义 Skill 实现盲道持续检测与环境拍照识别两大核心功能，并借助微信 ClawBot 消息通道，支持语音输入，极大地方便了用户/监护人的远程交互控制，构建了一个完整的"感知-决策-交互"视障辅助系统。

1.2 基本概念

1.2.1 实例分割

实例分割是计算机视觉中的一种任务，要求模型同时输出目标的检测框和像素级掩码（mask）。与目标检测相比，实例分割提供的像素级掩码能更精确地描述盲道的走向和宽度；与语义分割相比，实例分割可以区分同类别的不同实例。

本项目选用 YOLO26n-seg（YOLO 第 26 代 Nano 分割版本）作为基础模型，通过 tpu-mlir 工具链生成校准表后进行 int8 量化，最终部署到 Maixcam 上面。

量化过程中需要通过校准（Calibration）确定每层激活值的量化阈值。根据 TPU-MLIR 开发参考手册第 7.4 节，工具链实现了七种量化阈值计算方法。本项目使用 KLD（Kullback-Leibler Divergence）+ Auto-tune 算法进行校准：

KLD 算法：将 FP32 激活值的绝对值分布（2048 个 bin 的直方图）按不同截断位置（128 bin, 256 bin, ..., 2048 bin）截断，映射到 128 个 INT8 量化级别，计算截断后的 FP32 分布与扩展后的 INT8 分布之间的 KL 散度，选取散度最小的截断位置作为量化阈值。

Auto-tune 算法：在 KLD 阈值的基础上，逐层微调激活值的量化阈值。具体做法是：在 KLD 阈值和激活值最大绝对值之间均匀选取 20 个候选阈值，分别量化后计算输出与 FP32 参考输出的欧氏距离，选取欧氏距离最小的候选作为最终阈值。

1.2.2 PicoClaw Agent 框架

PicoClaw 是一个超轻量级 AI Agent 框架，核心特点包括：内存占用不足 10MB、单一自包含二进制文件、支持 RISC-V/ARM64/x86_64 多架构、支持飞书/钉钉/微信/Discord 等多种消息通道。其"Skill"机制允许开发者自定义功能模块，每个 Skill 可以独立处理特定任务并通过消息通道与用户交互。

本项目利用 PicoClaw（接入算能云算力平台 API）的 Skill 机制，在 MaixCam 端侧实现了两个自定义 Skill：

盲道检测 Skill（blind-lane-assistant）：封装完整的推理管线（图像采集→前处理→模型推理→后处理→方向判定），通过读取 status.json 中的 hint 值，将方向结果实时播报给用户。该 Skill 采用持续运行设计，服务永不停机（除非用户主动要求），且在意外退出时自动重启。

拍照识别 Skill（get-blind-photo）：当用户需要查看周围环境时触发，支持障碍物识别、红绿灯判断、纸币面额、文字朗读、场景描述等多种视觉分析功能。该 Skill 通过文件信号量机制（.snapshot_request → .snapshot_ready）向盲道检测服务请求按需快照，不中断检测服务、不杀进程，实现拍照与检测的无缝并行。

1.3 处理流程

模型训练转换及部署整体处理流程如下：



系统整体处理流程分为两条链路，由 PicoClaw Agent 统一调度：

链路一：盲道检测 Skill（持续运行）

摄像头实时采集 → 前处理（letterbox 缩放至 320×224）→ TPU INT8 推理 → 后处理（NMS + 掩码生成）→ 方向判定（掩码质心偏移）→ 状态写入 status.json → PicoClaw Agent 读取并播报方向

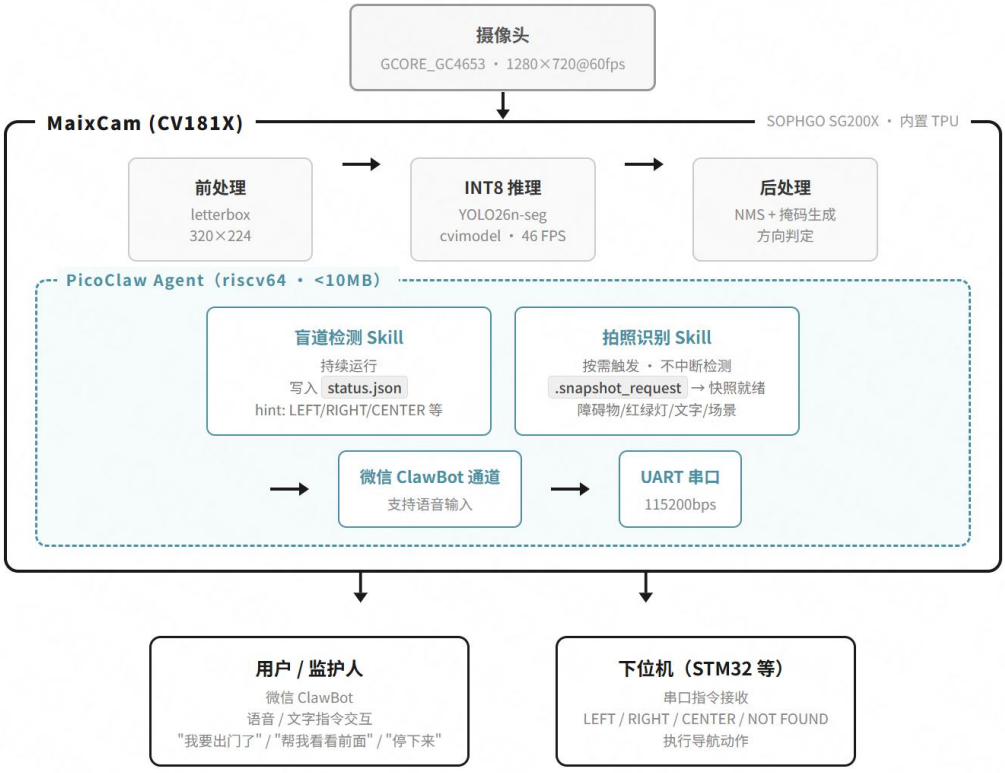
链路二：拍照识别 Skill（按需触发）

用户发送"帮我看看前面"等指令 → PicoClaw Agent 触发 get-blind-photo → 发送.snapshot_request 快照请求（不中断检测服务）→ 等待 .snapshot_ready → 读取 snapshot.jpg → 视觉分析 → 向用户返回结果

两条链路通过 PicoClaw Agent 的 Skill 调度机制实现协同。链路二采用文件信号量机制实现与链路一的并行运行，确保拍照识别过程中盲道检测不中断。

二、系统体系结构设计

2.1 系统架构



2.2 模块划分

模块	功能	实现方式
图像采集	获取摄像头实时画面	MaixPy camera 模块，1280×720@60fps
前处理	等比缩放+居中填充至 320×224	TPU-MLIR 内置前处理（letterbox）
模型推理	实例分割，输出检测框、类别、掩码	INT8 cvimodel 在 TPU 上执行
后处理	NMS 去重 + 掩码阈值化	Python 实现
方向判定	计算掩码质心偏移，判定盲道方向	规则引擎
串口通信	向下位机发送方向指令	UART 115200bps
PicoClaw	Skill 调度、消息路由、通道管理	PicoClaw riscv64 版本（内存 <10MB），MaixCam 本地运行
盲道检测 Skill	封装推理管线，持续检测并实时播报方向	自定义 Skill（Python），通过 status.json 输出 hint 值
拍照识别 Skill	按需触发拍照，障碍物/红绿灯/文字/场景识别	自定义 Skill（Python），文件信号量机制（.snapshot_request → .snapshot_ready）

微信通道	用户指令接收（支持语音）与结果消息推送	PicoClaw 通道适配，微信 ClawBot
JXA7T 离线语音播报模块	接收 MaixCam 经 UART 发送的方向指令，并播放对应语音	JXA7T 离线语音模块，UART115200 通信。

2.3 技术选型

技术环节	选型	理由
模型架构	YOLO26n-seg	Nano 版本轻量，支持实例分割
训练框架	Ultralytics	官方 YOLO 框架，训练导出一体化
模型转换	TPU-MLIR v1.27	SOPHGO 官方工具链，支持 CV181X
量化方案	INT8 对称量化	CV181X 支持 BF16 和 INT8
部署平台	MaixCam (CV181X)	SOPHGO SG200X 系列，内置 TPU
推理库	MaixPy nn 模块	MaixPy 原生推理接口
通信方式	UART 串口	简单可靠，115200bps
Agent 框架	PicoClaw	超轻量（<10MB），原生支持 MaixCam RISC-V，支持 Skill 扩展
交互通道	微信 ClawBot	用户基数大，支持语音输入，操作门槛低，适合视障人士及监护人使用
语音方案	JXA7T 离线语音模块	无需 STM32 及外置功放即可完成语音播报，降低系统复杂度，提高集成度，降低功耗。

2.4 接口描述

模型输入接口：

参数	值
输入尺寸	[1, 3, 224, 320]（NCHW）
像素格式	rgb
前处理	mean=0.0, scale=1/255, keep_aspect_ratio, letterbox 居中填充

模型输出接口：

输出节点	形状	含义
cv2.0 / cv2.1 / cv2.2	(1,4,H,W)	3 个尺度的检测框回归

输出节点	形状	含义
cv4.0 / cv4.1 / cv4.2	(1,32,H,W)	3 个尺度的原型特征
cv3.0 / cv3.1 / cv3.2	(1,1,H,W)	3 个尺度的掩码系数
output1	(1,32,56,80)	全局分割输出

串口通信协议（参考 MaixPy UART 文档）：

功能	发送数据（HEX）	说明
LEFT	AA 55 01 55 AA	播报"向左调整"
RIGHT	AA 55 02 55 AA	播报"向右调整"
CENTER	AA 55 03 55 AA	播报"保持直行"
CENTER-LEFT	AA 55 04 55 AA	略偏左
CENTER-RIGHT	AA 55 05 55 AA	略偏右
WEAK	AA 55 06 55 AA	检测信号较弱
NOT FOUND	AA 55 07 55 AA	未检测到盲道

PicoClaw Skill 接口：

Skill	触发方式	输入	输出
盲道检测 Skill（blind-lane-assistant）	用户语音/文字"我要出门了"	摄像头实时画面	方向播报（hint 值）
拍照识别 Skill（get-blind-photo）	用户语音/文字"帮我看看前面"等	按需快照 snapshot.jpg	障碍物/红绿灯/文字/场景识别结果

微信 ClawBot 控制指令：

用户指令（语音/文字）	响应动作
我要出门了 / 开始导航	启动盲道检测 Skill
停下来 / 到了 / 结束	停止盲道检测 Skill
帮我看看前面 / 前面有什么	触发拍照识别 Skill，分析当前画面
这是红灯还是绿灯	触发拍照识别 Skill，判断红绿灯

用户指令（语音/文字）	响应动作
这上面写的什么	触发拍照识别 Skill，识别文字

三、详细设计与实现

3.1 开发板接线说明

MaixCam UART TX→JXA7T RX；MaixCam UART RX→JXA7T TX；双方 GND 共地；JXA7T SPK+、SPK−连接扬声器。系统取消 STM32 及外置功放，由 JXA7T 直接完成语音播放。

摄像头使用 MaixCam 板载 MIPI 接口，型号 GCORE_GC4653，支持 1280×720@60fps。

3.2 软件流程图



3.3 关键代码分析

3.3.1 模型训练

```
from ultralytics import YOLO
model = YOLO("yolo26n-seg.pt")   # 加载预训练模型
results = model.train(
    data="blind_lane.yaml",        # 数据集配置
    epochs=50,                     # 训练轮次
    batch=32,                      # 批大小
    imgsz=[320, 224],              # 输入分辨率（宽×高）
)
```

```

### 3.3.2 ONNX 导出

```
from ultralytics import YOLO
model = YOLO("best.pt")
model.export(
    format="onnx",
    imgsz=[320, 224],   # 非正方形输入，适配道路场景横向视野
    batch=1,
    dynamic=False,   # 端侧要求固定输入尺寸
    simplify=True,   # 融合冗余算子节点
    half=False,
    nms=False,   # NMS 非标准算子，后处理自行实现
    opset=12,
)
```

### 3.3.3 ONNX → MLIR 转换

根据 TPU-MLIR 开发参考手册 Table 3.1（model_transform.py 参数表），执行以下命令：

```bash
model_transform.py \
    --model_name yolo26n_seg \
    --model_def 224_320.onnx \
    --input_shapes [[1,3,224,320]] \
    --mean 0.0,0.0,0.0 \
    --scale 0.0039215686,0.0039215686,0.0039215686 \
    --keep_aspect_ratio \
    --pixel_format rgb \
    --output_names "output1,/model.23/one2one_cv4.0/one2one_cv4.0.2/Conv_output_0,\
    /model.23/one2one_cv4.1/one2one_cv4.1.2/Conv_output_0,\
    /model.23/one2one_cv4.2/one2one_cv4.2.2/Conv_output_0,\
    /model.23/one2one_cv2.0/one2one_cv2.0.2/Conv_output_0,\
    /model.23/one2one_cv2.1/one2one_cv2.1.2/Conv_output_0,\
    /model.23/one2one_cv2.2/one2one_cv2.2.2/Conv_output_0,\
    /model.23/one2one_cv3.0/one2one_cv3.0.2/Conv_output_0,\
    /model.23/one2one_cv3.1/one2one_cv3.1.2/Conv_output_0,\
    /model.23/one2one_cv3.2/one2one_cv3.2.2/Conv_output_0" \
    --test_input test.jpg \
    --test_result yolo26n_seg_top_outputs.npz \
    --mlir yolo26n_seg.mlir
```

关键参数说明
```

参数	值	手册说明
keep_aspect_ratio	true	缩放时是否保持宽高比，默认为 false；开启后不足部分用 0 填充
pixel_format	rgb	图像类型，可选 rgb、bgr、gray 或 rgbd，默认为 bgr
mean	0.0, 0.0, 0.0	图像各通道的均值，默认为 0.0, 0.0, 0.0, 0.0
scale	1/255	图像各通道的缩放系数

此步骤同时生成 `yolo26n_seg_in_f32.npz`（测试输入）和 `yolo26n_seg_top_outputs.npz`（FP32 参考输出），作为后续量化精度对比的基准。

3.3.4 BF16 模型生成

根据 TPU 手册可知 CV181X 支持的量化类型为 BF16 和 INT8：

```
model_deploy.py \
    --mlir yolo26n_seg.mlir \
    --quantize bf16 \
    --processor cv181x \
    --test_input test.jpg \
    --test_reference yolo26n_seg_top_outputs.npz \
    --tolerance 0.98,0.80 \
    --model yolo26n_seg_bf16.cvimodel
```

3.3.5 INT8 校准与模型生成

根据 TPU 手册 Table 3.2（run_calibration.py 参数表）：

```
# 生成校准表
run_calibration.py yolo26n_seg.mlir \
    --dataset calibration/ \
    --input_num 100 \
    -o yolo26n_seg_table
```

根据 TPU 手册 Table 3.3，生成 INT8 模型：

```
model_deploy.py \
    --mlir yolo26n_seg.mlir \
    --quantize int8 \
    --quant_input \
    --processor cv181x \
    --calibration_table yolo26n_seg_table \
    --test_input test.jpg \
    --test_reference yolo26n_seg_top_outputs.npz \
    --tolerance 0.98,0.80 \
    --model yolo26n_seg_int8.cvimodel
```

其中 `--quant_input` 参数根据手册 Table 3.3 的说明为"Strip input type cast in bmodel,

need outside type conversion"。实测表明，指定该参数后 INT8 模型帧率从 13.6 提升至 46.0 FPS（详见 4.4 节）。

3.3.6 端侧推理核心代码

模型有 10 个输出节点，需要按通道数自动分类到检测框、类别分数、掩码系数、原型特征

```python
# detect() 方法中的输出分类逻辑
for _, t in output_iter:
    arr = tensor.tensor_to_numpy_float32(t, copy=False)
    arr_nchw = self._to_nchw(arr)
    c = int(arr_nchw.shape[1])
    h = int(arr_nchw.shape[2])
    w = int(arr_nchw.shape[3])
    key = (h, w)

    if c == 4:
        box_map[key] = arr_nchw       # 检测框回归 (1,4,H,W)
    elif c == 32:
        c32_list.append(arr_nchw)     # 原型特征 或 掩码系数
    else:
        cls_map[key] = arr_nchw       # 类别置信度 (1,1,H,W)

# 区分 32 通道中哪些是掩码系数（与检测框同尺度），哪个是原型特征（最大面积）
for t in c32_list:
    h, w = int(t.shape[2]), int(t.shape[3])
    key = (h, w)
    if key in box_map:
        coef_map[key] = t             # 与检测框同尺度 → 掩码系数
    else:
        proto = t                     # 最大面积的 32 通道 → 原型特征图

```

将原始输出解码为候选检测框。fast_single_class=True 时只取最高置信度位置，避免遍历所有网格点：

```python
def _decode_head_fast_single(self, cls_nchw, box_nchw, coef_nchw,
                             stride_x, stride_y, conf_th, out):
    cls_map = cls_nchw[0]   # (1, H, W)
    box_map = box_nchw[0]   # (4, H, W)
    coef_map = coef_nchw[0] # (32, H, W)
    cc, h, w = cls_map.shape

    # 对类别分数做 sigmoid，找最高置信度位置
    score_map = 1.0 / (1.0 + np.exp(-cls_map[0]))
    flat_scores = score_map.reshape(-1)
    best_idx = int(np.argmax(flat_scores))
    best_score = float(flat_scores[best_idx])
    if best_score < conf_th:
        return

    gy = best_idx // w
    gx = best_idx % w

    # 解码检测框：l,t,r,b 是到网格中心的距离（乘以 stride 还原到输入尺寸）
    l = float(box_map[0, gy, gx])
    t = float(box_map[1, gy, gx])
    r = float(box_map[2, gy, gx])
    b = float(box_map[3, gy, gx])

    cx = (gx + 0.5) * stride_x
    cy = (gy + 0.5) * stride_y

    x = int(cx - l * stride_x)
    y = int(cy - t * stride_y)
    bw = int((l + r) * stride_x)
    bh = int((t + b) * stride_y)

    out.append(Candidate(
        x=x, y=y, w=bw, h=bh,
        score=best_score, class_id=0,
        mask_coefs=coef_map[:, gy, gx].copy(),  # 取该位置的 32 维掩码系数
    ))

```

将掩码系数与原型特征做矩阵乘，经 sigmoid 和阈值化生成像素级掩码，并有多级降级策略：

```python
# detect() 方法中的掩码生成逻辑
proto_map = proto[0]          # (32, mask_h, mask_w)
mask_h = int(proto_map.shape[1])
mask_w = int(proto_map.shape[2])

for cand in final_candidates:
    # 将检测框坐标映射到原型特征图的区域
    x1 = max(0, cand.x)
    y1 = max(0, cand.y)
    x2 = min(self.input_w, cand.x + cand.w)
    y2 = min(self.input_h, cand.y + cand.h)

    mx1 = max(0, x1 * mask_w // self.input_w)
    my1 = max(0, y1 * mask_h // self.input_h)
    mx2 = min(mask_w, x2 * mask_w // self.input_w)
    my2 = min(mask_h, y2 * mask_h // self.input_h)

    # 裁剪原型特征图的对应区域
    proto_crop = proto_map[:, my1:my2, mx1:mx2]
    coef = cand.mask_coefs.reshape(32, 1, 1)

    # 核心计算：掩码系数 × 原型特征 → sigmoid → 二值化
    small_logits = np.sum(proto_crop * coef, axis=0)
    small_prob = 1.0 / (1.0 + np.exp(-small_logits))
    small_mask = np.where(small_prob > 0.50, 255, 0).astype(np.uint8)

    # 多级降级策略：避免 INT8 量化后掩码为空
    if np.count_nonzero(small_mask) == 0:
        small_mask = np.where(small_prob > 0.35, 255, 0).astype(np.uint8)
    if np.count_nonzero(small_mask) == 0:
        dyn_th = float(np.max(small_prob)) * 0.70
        small_mask = np.where(small_prob >= dyn_th, 255, 0).astype(np.uint8)
    if np.count_nonzero(small_mask) == 0:
        small_mask = np.full((crop_h, crop_w), 255, dtype=np.uint8)

    # 缩放回原图坐标
    resized_mask = self._resize_nearest(small_mask, orig_w, orig_h)

```

3.3.7 串口通信代码

根据 MaixPy UART 文档：

```
from maix import uart, pinmap
# 设置引脚功能
pinmap.set_pin_function("A16", "UART0_TX")
pinmap.set_pin_function("A17", "UART0_RX")
# 初始化串口
serial = uart.UART("/dev/ttyS0", 115200)
# 发送方向指令
CMD_MAP = {
    "LEFT": b'\xAA\x55\x01\x55\xAA',
    "RIGHT": b'\xAA\x55\x02\x55\xAA',
    "CENTER": b'\xAA\x55\x03\x55\xAA',
    "CENTER-LEFT": b'\xAA\x55\x04\x55\xAA',
    "CENTER-RIGHT": b'\xAA\x55\x05\x55\xAA',
    "WEAK": b'\xAA\x55\x06\x55\xAA',
```

```
    "NOT FOUND": b'\xAA\x55\x07\x55\xAA',
}
serial.write(CMD_MAP["CENTER"])
```

为提高串口通信的可靠性，本系统未采用 ASCII 字符串协议，而是设计了固定长度的二进制通信协议。每条指令均以 0xAA55 为帧头、0x55AA 为帧尾，中间使用 1 Byte 指令码表示不同播报内容。JXA7T 语音模块接收到对应字节序列后，立即播放预先存储的语音，实现低延迟、低误码率的语音提示。

3.4 PicoClaw 部署以及配置

3.4.1 部署 picoclaw



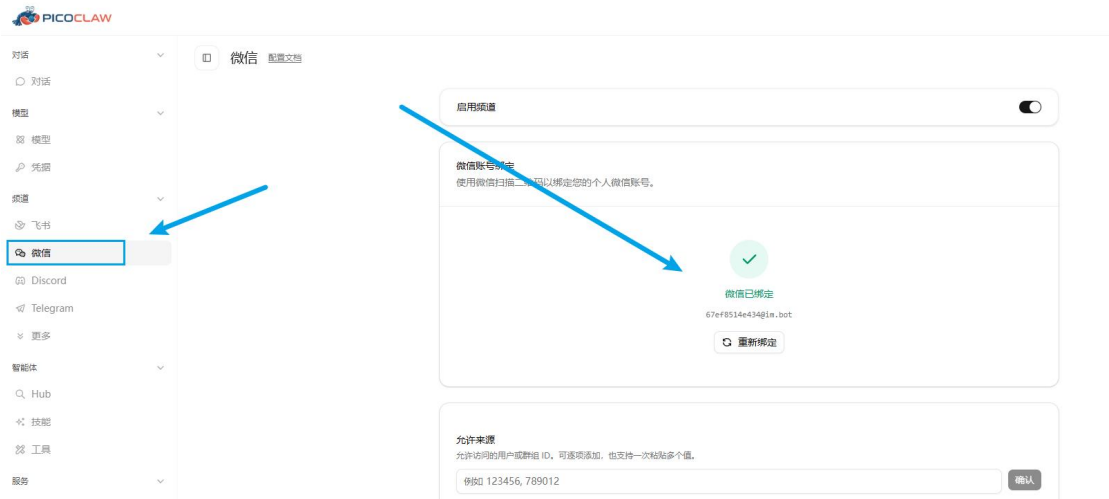
如图所示：


kill 294
picoclaw-launcher -no-browser -public -console
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:430 > PicoClaw launcher starting (version 0.2.8)...
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:431 > PicoClaw Home: /root/.picoclaw
PICOCLAW
Dashboard address:
>> http://localhost:18800 <<
>> http://10.157.146.1:18800 <<
>> http://10.157.145.1:18800 <<
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:637 > Server will listen on http://[::]:18800
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:637 > Server will listen on http://0.0.0.0:18800
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:641 > Public access enabled at http://10.157.146.1:18800
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:664 > Server listening on [::]:18800
01:05:32 INF web ../home/runner/work/picoclaw/picoclaw/web/backend/main.go:664 > Server listening on 0.0.0.0:18800
01:05:33 INF gateway ../home/runner/work/picoclaw/picoclaw/web/backend/api/gateway.go:879 > Starting gateway p

配置算能云平台 API


配置 Qwen3-VL-235B-A22B-Instruct
Qwen3-VL-235B-A22B-Instruct
Provider
可选。指定后，将以该值作为最终 provider，并将"模型标识符"字段解释为规范模型 ID。
openai
模型标识符
未指定 Provider 时，诸如 openai/gpt-4o 的值将按 provider/model 格式解析。已指定 Provider 时，此字段将作为规范模型 ID 使用，不再解析其中的 provider 前缀。
Qwen3-VL-235B-A22B-Instruct
API Key
已设置 API Key，留空表示不修改。
6Z****9Q
API Base URL
https://www.sophnet.com/api/open-apis/v1
默认模型
保存后自动将该模型设置为默认模型。
高级选项
取消
保存

配置微信通道：


PICOCLAW
对话
模型
凭据
频道
飞书
微信
Discord
Telegram
更多
智能体
Hub
技能
工具
服务
微信 配置文档
启用频道
微信已绑定
重新绑定
允许来源
确认

3.4.2 盲道检测 Skill 实现（blind-lane-assistant)

盲道检测 Skill 封装了完整的推理管线，采用持续运行设计。通过 nohup 后台启动 blind_service_int8.py，防止会话超时导致服务被杀。服务永不停机（除非用户主动要求），即使检测到 NOT FOUND 也不停止。如果服务意外退出，Skill 自动检测进程并重新启动。

3.4.3 拍照识别 Skill 实现（get-blind-photo)

拍照识别 Skill 通过文件信号量机制与盲道检测服务协同，不中断检测服务。核心流程：创建 .snapshot_request 文件触发快照 → 轮询等待 .snapshot_ready 出现（最多 3 秒）→ 读取 snapshot.jpg 进行视觉分析 → 返回结果。支持障碍物识别、红绿灯判断、纸币面额、文字朗读、场景描述等多种场景。

四、系统测试与分析

4.1 测试环境

项目	说明
开发板	MaixCam（SOPHGO SG200X / CV181X）
摄像头	GCORE_GC4653_MIPI_4M_30FPS_10BIT（1280×720@60fps）
模型输入尺寸	320×224
测试方法	预热 10 帧 + 正式测试 100 帧
推理库	MaixPy nn 模块
转换工具	TPU-MLIR v1.27-20260206

4.2 量化精度验证(附录日志可查看)

TPU-MLIR 工具链在生成最终 cvimodel 前，自动进行了两轮精度验证。
第一轮：MLIR 量化输出 vs FP32 参考输出
验证标准：余弦相似度 ≥ 0.98，欧氏相似度 ≥ 0.80

输出节点	形状	余弦相似度	欧氏相似度	SQNR (dB)
cv2.0（检测框回归，28×40）	(1,4,28,40)	0.9941	0.8910	14.72
cv4.0（原型特征，28×40）	(1,32,28,40)	0.9967	0.9178	21.70
cv3.0（掩码系数，28×40）	(1,1,28,40)	0.9990	0.9550	11.42
cv2.1（检测框回归，14×20）	(1,4,14,20)	0.9980	0.9368	18.59
cv4.1（原型特征，14×20）	(1,32,14,20)	0.9964	0.9140	21.12
cv3.1（掩码系数，14×20）	(1,1,14,20)	0.9981	0.9351	4.36
cv2.2（检测框回归，7×10）	(1,4,7,10)	0.9990	0.9551	21.17
cv4.2（原型特征，7×10）	(1,32,7,10)	0.9986	0.9478	25.22

cv3.2（掩码系数，7×10）	(1,1,7,10)	0.9953	0.8800	8.25
output1（全局分割输出）	(1,32,56,80)	0.9928	0.8785	17.10

10 个输出节点全部 PASSED，最低余弦相似度 0.9928（阈值 0.98）

第二轮：最终 cvimodel vs MLIR 量化输出

更严格验证（余弦相似度 ≥ 0.99，欧氏相似度 ≥ 0.90），确认 codegen 生成的最终模型文件正确性：

输出节点	余弦相似度	欧氏相似度	SQNR (dB)	结果
cv2.0	0.9993	0.9616	23.74	PASSED
cv4.0	0.9995	0.9684	29.94	PASSED
cv3.0	0.9999	0.9865	21.88	PASSED
cv2.1	0.9994	0.9665	23.98	PASSED
cv4.1	0.9992	0.9589	27.46	PASSED
cv3.1	0.9998	0.9811	14.68	PASSED
cv2.2	0.9997	0.9747	26.07	PASSED
cv4.2	0.9996	0.9713	30.41	PASSED
cv3.2	0.9996	0.9729	20.83	PASSED
output1	0.9982	0.9395	23.21	PASSED

10 个节点全部 PASSED，最低余弦相似度 0.9982，最低欧氏相似度 0.9395，均大幅超过阈值。

BF16 精度验证（作为精度基准对照）

第一轮验证（MLIR 量化输出 vs FP32 参考输出，阈值 cosine ≥ 0.98）：

输出节点	余弦相似度	欧氏相似度	SQNR (dB)	结果
cv2.0	0.9999	0.9873	33.66	PASSED
cv4.0	0.9999	0.9895	39.53	PASSED
cv3.0	1.0000	0.9923	29.19	PASSED
cv2.1	1.0000	0.9921	36.66	PASSED
cv4.1	0.9999	0.9865	37.16	PASSED
cv3.1	1.0000	0.9936	26.16	PASSED
cv2.2	1.0000	0.9937	38.11	PASSED
cv4.2	1.0000	0.9926	42.22	PASSED
cv3.2	0.9999	0.9873	29.87	PASSED
output1	0.9999	0.9814	34.01	PASSED

BF16 最低余弦相似度 0.9999，作为精度基准。

精度对比总结

版本	最低 cosine	最低 euclidean	最低 SQNR (dB)
BF16（基准）	0.9999	0.9814	26.16
INT8（量化）	0.9928	0.8785	4.36
差距	-0.0071	-0.1029	-21.80

INT8 量化后余弦相似度仅下降 0.0071，验证结果显示精度损失可忽略不计。

4.3 端侧推理性能测试

BF16 vs INT8 性能对比

测试项	BF16 模型	INT8 模型	变化
模型文件大小	5.67 MB	2.87 MB	-49.4%
模型加载耗时	46 ms	27 ms	-41.3%
平均 FPS	21.4	46.0	+114.9%（2.15 倍）
每帧平均耗时	46.7 ms	21.7 ms	-53.5%
每帧中位数耗时	47.0 ms	21.0 ms	-55.3%
推理平均耗时	45.1 ms	19.9 ms	-55.9%
推理中位数耗时	45.0 ms	19.0 ms	-57.8%
P95 帧耗时	53.0 ms	29.0 ms	-45.3%
P99 帧耗时	57.0 ms	34.0 ms	-40.4%

结果分析

推理速度：INT8 模型的平均帧率达到 46.0 FPS，是 BF16 模型（21.4 FPS）的 2.15 倍。单帧推理耗时从 45.1 ms 降至 19.9 ms，降幅达 55.9%。

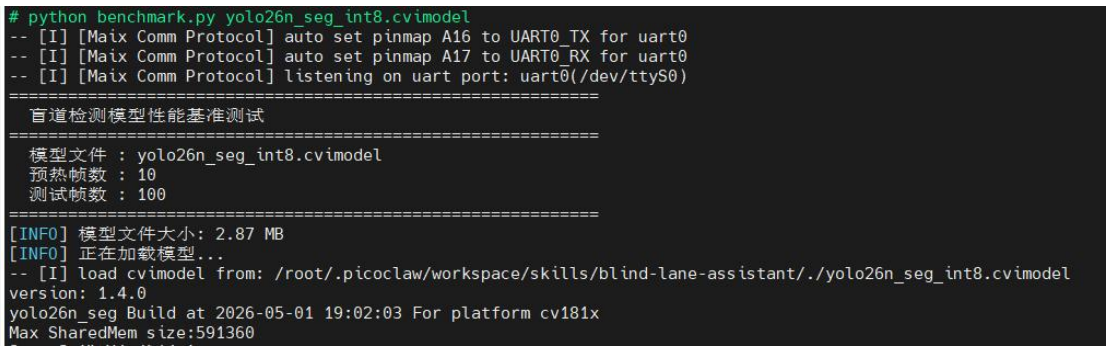
模型体积：INT8 量化将模型从 5.67 MB 压缩至 2.87 MB，减少 49.4%，接近理论值（BF16→INT8 权重部分减半）。

稳定性：INT8 模型的 P99 帧耗时（34.0 ms）仍低于 BF16 的中位数耗时（47.0 ms），说明 INT8 在最坏情况下也比 BF16 的典型表现更快，推理延迟稳定可控。

部署决策：综合速度和体积优势，选用 INT8 版本作为最终部署模型。在 320×224 分辨率下实现 46 FPS 的实时推理，远超 15 FPS 的实时性要求。

INT8 量化关键参数：--quant_input

在首次 INT8 转换中，未指定 `--quant_input` 参数，实测 INT8 模型（13.6 FPS）反而慢于 BF16（14.8 FPS）。加上该参数后，INT8 模型帧率从 13.6 提升至 46.0，提升 3.4 倍。根据手册 Table 3.3，`--quant_input` 的作用是"Strip input type cast in bmodel"，即去除模型内部的输入类型转换。实测表明，对于本项目的小模型，去除该固定开销对帧率影响显著。



```
[INFO] 开始正式测试（100 帧）...
  [20/100] 当前平均: 20.8 ms/帧 (48.1 fps)
  [40/100] 当前平均: 21.1 ms/帧 (47.5 fps)
  [60/100] 当前平均: 21.4 ms/帧 (46.7 fps)
  [80/100] 当前平均: 21.3 ms/帧 (46.9 fps)
  [100/100] 当前平均: 23.2 ms/帧 (43.1 fps)

============================================================
  测试结果: yolo26n_seg_int8.cvimodel
============================================================
  模型文件大小          : 2.87 MB
  模型输入尺寸          : 320x224
  模型加载耗时          : 431 ms
  实际测试帧数          : 100
  总测试时间            : 2.32 秒
------------------------------------------------------------
  平均 FPS              : 43.1
------------------------------------------------------------
  每帧总耗时（取帧+推理）
    平均                : 23.2 ms
    中位数              : 21.0 ms
    最小                : 19.0 ms
    最大                : 57.0 ms
    P95                 : 34.0 ms
    P99                 : 57.0 ms
------------------------------------------------------------
  推理耗时（前处理+TPU+后处理）
    平均                : 21.1 ms
    中位数              : 20.0 ms
    最小                : 17.0 ms
    最大                : 53.0 ms
------------------------------------------------------------
  取帧耗时
    平均                : 2.1 ms
============================================================


| 指标 | 值 |
|---|---|
| 模型文件 | yolo26n_seg_int8.cvimodel |
| 模型大小 | 2.87 MB |
| 输入尺寸 | 320x224 |
| 平均 FPS | 43.1 |
| 每帧平均耗时 | 23.2 ms |
| 每帧中位数耗时 | 21.0 ms |
| 推理平均耗时 | 21.1 ms |
| P95 帧耗时 | 34.0 ms |
| P99 帧耗时 | 57.0 ms |
```

```
# python benchmark.py yolo26n_seg_bf16.cvimodel
-- [I] [Maix Comm Protocol] auto set pinmap A16 to UART0_TX for uart0
-- [I] [Maix Comm Protocol] auto set pinmap A17 to UART0_RX for uart0
-- [I] [Maix Comm Protocol] listening on uart port: uart0(/dev/ttyS0)
============================================================
  盲道检测模型性能基准测试
============================================================
  模型文件 : yolo26n_seg_bf16.cvimodel
  预热帧数 : 10
  测试帧数 : 100
============================================================
[INFO] 模型文件大小: 5.67 MB
[INFO] 正在加载模型...
-- [I] load cvimodel from: /root/.picoclaw/workspace/skills/blind-lane-assistant/./yolo26n_seg_bf16.cvimodel
version: 1.4.0
yolo26n_seg Build at 2026-05-01 19:01:25 For platform cv181x
Max SharedMem size:1182720
```

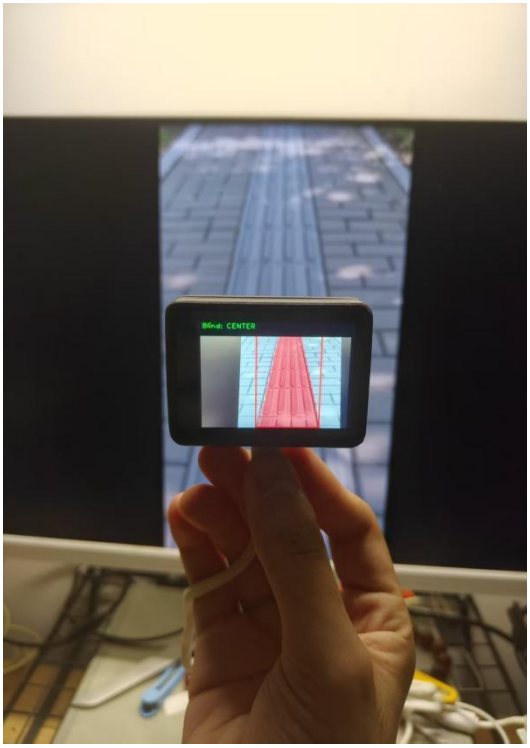
```
[INFO] 开始正式测试（100 帧）...
  [20/100] 当前平均: 57.0 ms/帧 (17.5 fps)
  [40/100] 当前平均: 51.7 ms/帧 (19.4 fps)
  [60/100] 当前平均: 50.1 ms/帧 (20.0 fps)
  [80/100] 当前平均: 49.7 ms/帧 (20.1 fps)
  [100/100] 当前平均: 50.1 ms/帧 (20.0 fps)

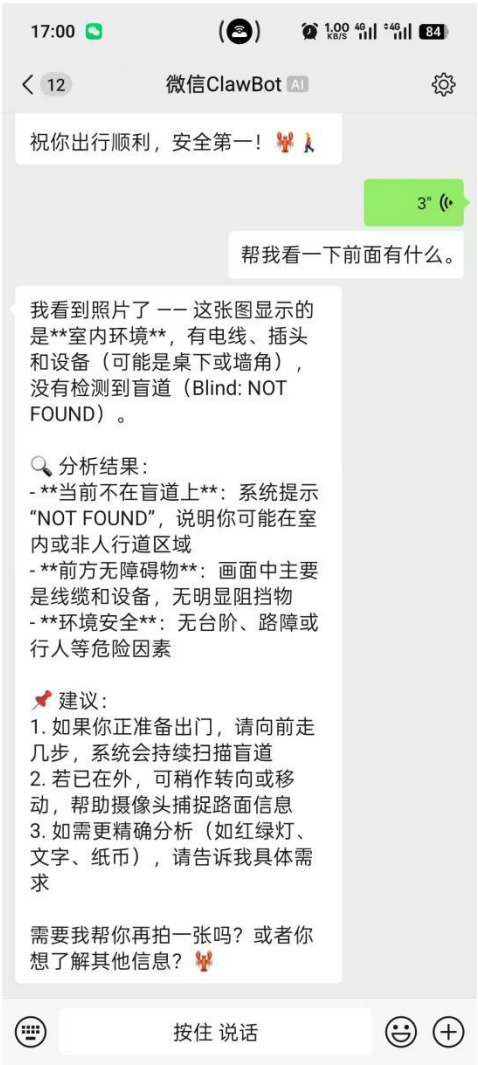
============================================================
  测试结果: yolo26n_seg_bf16.cvimodel
============================================================
  模型文件大小          : 5.67 MB
  模型输入尺寸          : 320x224
  模型加载耗时          : 632 ms
  实际测试帧数          : 100
  总测试时间            : 5.01 秒
------------------------------------------------------------
  平均 FPS              : 20.0
------------------------------------------------------------
  每帧总耗时（取帧+推理）
    平均                : 50.1 ms
    中位数              : 47.0 ms
    最小                : 43.0 ms
    最大                : 68.0 ms
    P95                 : 66.0 ms
    P99                 : 68.0 ms
------------------------------------------------------------
  推理耗时（前处理+TPU+后处理）
    平均                : 47.5 ms
    中位数              : 45.0 ms
    最小                : 42.0 ms
    最大                : 63.0 ms
------------------------------------------------------------
  取帧耗时
    平均                : 2.6 ms
============================================================


| 指标 | 值 |
|---|---|
| 模型文件 | yolo26n_seg_bf16.cvimodel |
| 模型大小 | 5.67 MB |
| 输入尺寸 | 320x224 |
| 平均 FPS | 20.0 |
| 每帧平均耗时 | 50.1 ms |
| 每帧中位数耗时 | 47.0 ms |
| 推理平均耗时 | 47.5 ms |
| P95 帧耗时 | 66.0 ms |
| P99 帧耗时 | 68.0 ms |
```

4.4 PicoClaw Agent 功能测试（相关 SKILL 内容在附录文件）


16:26
12
微信ClawBot AI
16:23
5"
我现在要出门了，请你帮我开一下盲道检测。
盲道检测已开启，可以出门了！
- 摄像头已启动，正在实时检测盲道
- 每15秒自动播报方向
- 当前状态：暂时没检测到盲道（正常，走起来就能看到了）
走路时我会持续提醒你方向，到了目的地或者想停下来的时候告诉我就行。注意安全！
盲道播报：暂时没看到盲道，注意脚下，慢慢走。服务正在持续检测中，有变化会立刻告诉你。
按住 说话


Blind: CENTER


17:00
12
微信ClawBot AI
祝你出行顺利，安全第一！
3"
帮我看一下前面有什么。
我看到照片了 —— 这张图显示的是**室内环境**，有电线、插头和设备（可能是桌下或墙角），没有检测到盲道（Blind: NOT FOUND）。
🔍 分析结果：
- **当前不在盲道上**：系统提示“NOT FOUND”，说明你可能在室内或非人行道区域
- **前方无障碍物**：画面中主要是线缆和设备，无明显阻挡物
- **环境安全**：无台阶、路障或行人等危险因素
📌 建议：
1. 如果你正准备出门，请向前走几步，系统会持续扫描盲道
2. 若已在外，可稍作转向或移动，帮助摄像头捕捉路面信息
3. 如需更精确分析（如红绿灯、文字、纸币），请告诉我具体需求
需要我帮你再拍一张吗？或者你想了解其他信息？
按住 说话

五、总结

5.1 主要工作

1. 模型训练：基于 YOLO26n-seg 在公开盲道数据集上训练实例分割模型，50 个 epoch 后 mAP50(M) 达到 0.785，精确率 0.918。
2. 模型转换：通过 TPU-MLIR v1.27 工具链完成 PyTorch → ONNX → MLIR → cvimodel 的完整转换链路，生成 BF16 和 INT8 两个版本。
3.自定义后处理实现：由于 TPU-MLIR 官方工具链尚未提供 YOLO26-seg 实例分割模型的后处理教程，本项目自行实现了完整的后处理链路，包括：多输出节点自动分类路由（按通道数区分检测框、类别分数、掩码系数和原型特征）、检测头解码（sigmoid 激活 + 网格坐标还原）、NMS 去重、像素级掩码生成（掩码系数与原型特征矩阵乘 + sigmoid + 多级阈值降级策略）、以及基于掩码质心的盲道方向判定算法（下半区域裁剪 + 左中右三等分投票 + 全局偏移比计算）。
4. INT8 量化优化：通过校准量化和 `--quant_input` 参数，在精度几乎无损的前提下（余弦相似度 ≥ 0.9928），实现推理速度 2.15 倍提升和模型体积 49.4% 压缩。
5.端侧部署：在 MaixCam 上实现 46 FPS 的实时盲道分割推理，满足实际导航场景的实时性需求。
6.PicoClaw Agent 部署：在 MaixCam（RISC-V）平台上成功部署 PicoClaw 超轻量 AI Agent 框架，接入 sophnet 云算力平台大模型 API，配置微信 ClawBot 消息通道，支持语音输入，实现端侧 Agent 能力。
7. 自定义 Skill 开发：实现"盲道检测 Skill"（blind-lane-assistant）和"拍照识别 Skill"（get-blind-photo）两个自定义 Skill，分别覆盖持续导航和环境识别两大视障出行核心场景。
8.微信远程交互：配置微信 ClawBot 通道，实现用户/监护人通过微信语音或文字远程控制检测启停、接收方向播报和环境识别结果。

5.2 关键技术贡献

贡献	说明
INT8 量化优化	利用 --quant_input 去除输入类型转换开销，实测速度提升 3.4 倍
后处理代码撰写	利用 Maixcdk 和 Maixpy，撰写两种版本的后处理代码 （Sipeed 官方未支持的 API）
量化精度保障	两轮自动精度验证（cosine ≥ 0.98 / 0.99），确保量化无损
端侧实时推理	46 FPS @ 320×224，远超 15 FPS 的实时要求
轻量化部署	模型仅 2.87 MB，适合资源受限的端侧设备
PicoClaw 部署	在 MaixCam RISC-V 平台上集成 PicoClaw，实现端侧 AI Agent 能力，内存占用 <10MB
双 Skill 协同架构	盲道检测 Skill + 拍照识别 Skill，通过文件信号量机制实现不中断检测的按需快照
微信 ClawBot 远程交互	支持语音输入，用户/监护人通过微信实时控制和接收信息，降低使用门槛

5.3 未来展望

1. 输入分辨率优化：测试不同输入分辨率（如 640×480）下的精度-速度权衡
2. 后处理融合：4 月 14 日，TPU-MLIR v1.28 更新日志中提到的 yolov26 后处理融合支持，或许推理处理速度会更好
3. 多传感器融合：集成超声波和 IMU，实现视觉+距离+运动状态的多模态感知
4. 进一步扩展 picoclaw agent 能力，同更多外设结合，优化内存。

六、AI 工具使用声明

由于我们团队并不是深度学习过神经网络知识，偏向于嵌入式开发，故而本项目有较多部分内容都是在 AI 的帮助下进行的，以下是使用 AI 工具的场景：

1、使用 Gemini 3.1 Pro 用于询问选取选取的量化节点是否可以加速处理，后处理代码辅助撰写。

2、使用小米的 Mimo 2.5Pro 辅助报告文档撰写。

3、使用 ChatGPT 5.3 Codex 辅助后处理代码优化。

4、AI 生成内容占比约为 38%左右，其中的核心工作（模型选型、训练调参、TPU-MLIR 转换流程、量化策略选择、硬件部署调试、串口通信联调），PCB 扩展板等等则均由本团队独立完成。